Part 2: Face Morphing Attack Detection

1. Chosen Article

Venkatesh, S.; Raghavendra, R.; Raja, K.; Busch, C. "Single Image Face Morphing Attack Detection Using Ensemble of Features." Proc. IEEE 23rd International Conference on Information Fusion (FUSION), 2020. This is a Single-image Morphing Attack Detection (S-MAD) method: it decides bona fide vs. morphed from one image alone, with no trusted live-capture reference -- the right fit for our setup, since Part 1 only produced the two source photos that were morphed together, not a separate trusted capture of either subject.

2. Method (as implemented)

Each face image is represented in two color spaces (YCbCr, HSV, 6 channel images total). Each channel gets a 3-level Laplacian pyramid to expose morphing residue at multiple scales (18 scale-space images per face). Three texture descriptors are computed on every scale-space image and concatenated per descriptor type across all 18: Local Binary Patterns (LBP, block-wise), Histogram of Oriented Gradients (HOG), and Binarized Statistical Image Features (BSIF). Each of the 3 resulting streams is classified independently by a Collaborative Representation Classifier (CRC): a probe vector is reconstructed as a ridge-regularized combination of each class's training vectors, and the class with lower reconstruction residual wins. The per-stream scores are z-score normalized against their own training distribution, then fused by summation.

src/mad_features.py, src/mad_crc.py -- pipeline core:

```
def scale_space_images(img_bgr):  # 6 color channels x 3 Laplacian levels
    return [_resize(level) for ch in to_color_channels(img_bgr)
                           for level in laplacian_pyramid(ch)]

class CRCClassifier:
    def fit(self, X, y):        # dictionary D_c = training vectors of class c
        D = Xn[y == c].T; self.proj_[c] = inv(D.T @ D + lam*I) @ D.T
    def score(self, X):         # residual(bona fide) - residual(morph)
        return [self._residual(x, 0) - self._residual(x, 1) for x in Xn]
```

3. Adaptations from the Paper

- BSIF normally uses a filter bank pre-learned via ICA on ~50,000 natural-image patches (Kannala & Rahtu, 2012). We do not have that corpus, so we learn a smaller ICA filter bank (6 filters, 5x5) directly from each cross-validation fold's own bona fide training images instead -- same principle, much smaller and fold-local training data.
- The paper evaluates digital and print-scanned morphs on 1,309 bona fide / 2,608 morphed images with a single fixed identity-disjoint train/test split. Our Part 1 dataset has 116 bona fide / 58 morphed digital images only, so we use 5-fold pair-disjoint cross-validation (GroupKFold on the source-pair id) to get an out-of-fold score for every image while still preventing any image pair from straddling train and test.
- We z-score each descriptor stream's scores against its own training distribution before the sum-rule fusion; the original paper does not state whether it normalizes before summing, and skipping this step let BSIF's very high-dimensional (73,728-d) residual scale dominate the fused score in our own initial run.

4. Dataset and Evaluation Protocol

Bona fide class: the 116 original LFW source images (Subject A and B) from Part 1's 58 pairs. Attack class: the 58 corresponding alpha=0.5 morphs. Evaluated with 5-fold pair-disjoint cross-validation and reported with the ISO/IEC 30107-3 metrics: APCER (attacks classified as bona fide), BPCER (bona fide classified as attacks), and D-EER (operating point where the two are equal).

5. Results

Configuration	D-EER (%)
LBP stream alone	18.97
HOG stream alone	18.97
BSIF stream alone	43.10
Fused ensemble (LBP+HOG+BSIF)	17.24

At the ISO/IEC 30107-3 operating points: BPCER = 57.76% at APCER = 5%, and BPCER = 41.38% at APCER = 10%.

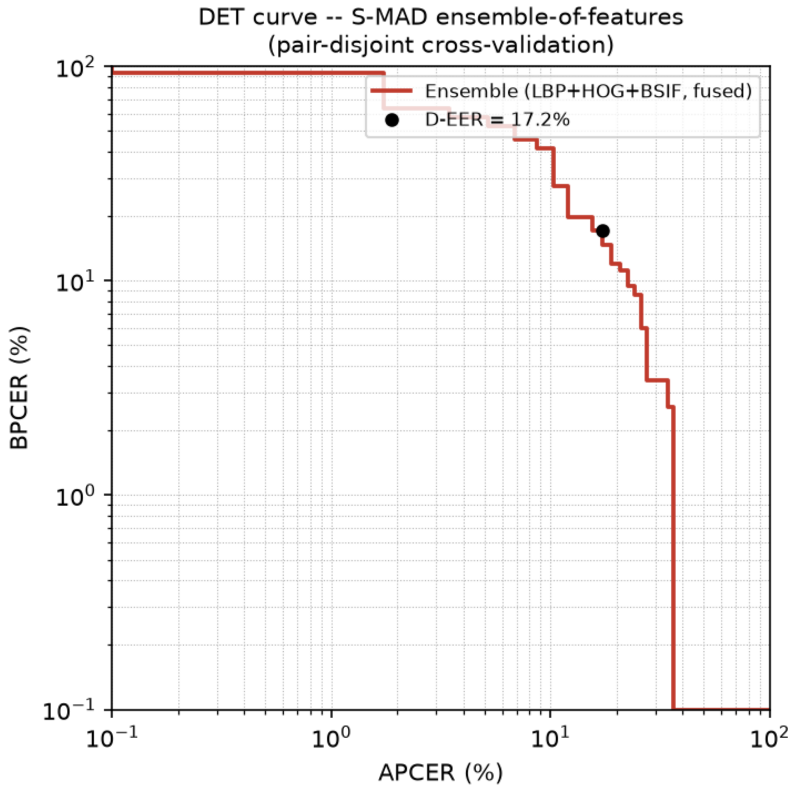

Figure 1: DET curve of the fused ensemble score, pair-disjoint 5-fold cross-validation (both axes log-scaled per ISO/IEC 30107-3 convention).

6. Observations and Limitations

- The fused ensemble (D-EER 17.2%) is noticeably weaker than the paper's own reported 5.99-5.64% D-EER, mainly because our training set is two orders of magnitude smaller (174 vs 3,900+ images) and identity-disjoint cross-validation on 58 pairs gives coarse-grained, high-variance error estimates.
- BSIF performed close to chance alone (43.1% D-EER) despite contributing to the best fused score, which we attribute to the filter-bank substitution (Adaptation #1): fold-local ICA on a few dozen bona fide images is a much weaker texture prior than filters learned on 50,000 diverse natural-image patches.
- LBP and HOG carried almost all of the usable signal (~19% D-EER each); fusing all three still beat either stream alone, showing the ensemble/sum-fusion idea holds even when one stream is individually weak.
- Our attack set only contains alpha=0.5 morphs from a single morphing pipeline (our own Part 1 Delaunay/warp code); a detector trained here is not shown to generalize to other morphing tools or alpha values, mirroring the generalization gap this literature repeatedly flags as an open problem.

7. Neural Network Fusion Layer

The paper combines the three descriptor-stream scores with a fixed, hand-picked sum rule. We additionally replace that fixed rule with the simplest possible neural network -- a single trainable neuron (logistic regression) -- that learns how much to weight each stream instead: z = w_lbp*lbp + w_hog*hog + w_bsif*bsif + b, p = sigmoid(z), trained by mini-batch stochastic gradient descent (batch size 8, 1000 iterations) on the binary cross-entropy loss. Its 3 inputs are the same z-scored, out-of-fold CRC scores from Section 5/6 -- already leakage-free -- so this experiment only needed one further pair-disjoint 80/20 train/test split (138 training images / 36 test images) to fit and evaluate the neuron itself.

src/mad_nn.py -- the fusion neuron:

```
class LogisticNeuron:
    def fit(self, X, y, X_val, y_val):   # X = [lbp_score, hog_score, bsif_score] per image
        for _ in range(n_iterations):
            xb, yb = mini_batch(X, y)          # random mini-batch each step
            p_batch = sigmoid(xb @ self.weights + self.bias)
            self.weights -= lr * xb.T @ (p_batch - yb) / batch_size   # gradient descent
            self.bias    -= lr * mean(p_batch - yb)
            # log train/test accuracy+loss and the weights/bias at this step
```

Metric	Value
Training accuracy	92.03%
Test accuracy	88.89%
Final training cross-entropy loss	0.3505
Final test cross-entropy loss	0.3582
Weight - LBP stream	4.874
Weight - HOG stream	7.984
Weight - BSIF stream	-1.580
Bias	-3.437

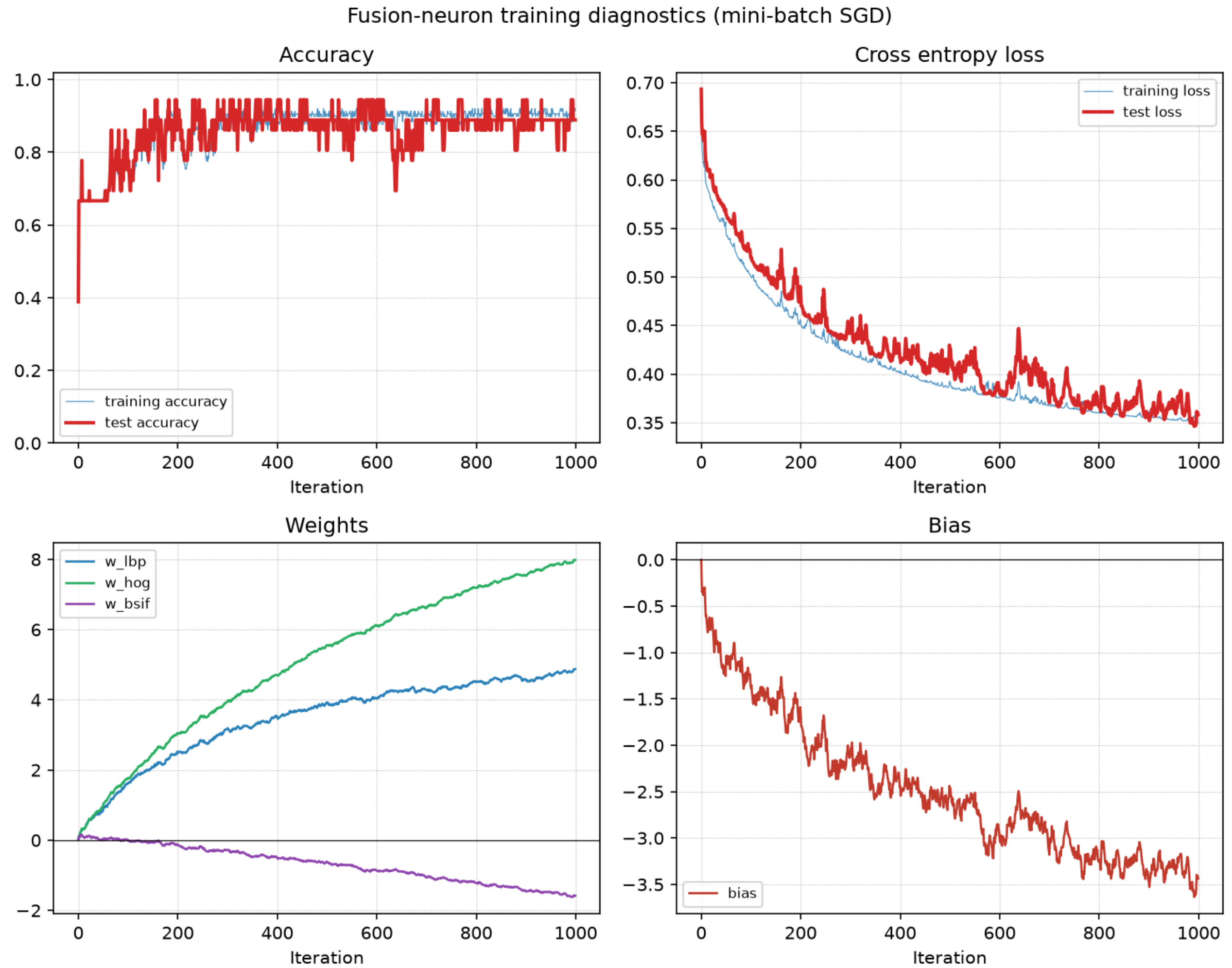

Figure 2: Training diagnostics of the fusion neuron over 1000 mini-batch SGD iterations (learning rate 0.5) -- accuracy, cross-entropy loss, and the weight/bias trajectories for train (thin blue) vs. held-out test (thick red) data.

The learned weights corroborate Section 6's finding purely from the training data, with no hand-tuning: LBP and HOG get large positive weights that keep growing (the neuron relies on them heavily), while BSIF's weight drifts negative -- the network learned on its own that the BSIF stream carries little useful signal, consistent with its near-chance 43.1% standalone D-EER.

8. Conclusion

We reimplemented the ensemble-of-features S-MAD pipeline from Venkatesh et al. (FUSION 2020) end-to-end on our own bona fide/morph dataset from Part 1: color-space expansion, Laplacian-pyramid scale space, LBP/HOG/BSIF descriptors, per-stream P-CRC classification, and sum-rule fusion, evaluated with the ISO/IEC 30107-3 metrics the field standardizes on. The pipeline detects our own morphs meaningfully better than chance (D-EER 17.2%), with the expected accuracy gap to the original paper explained by dataset scale and our BSIF filter-bank substitution, both documented above. We further trained a small neural network (a single logistic-regression neuron) to learn the stream-fusion weights directly from data instead of a fixed sum rule, reaching 88.9% test accuracy and independently confirming, through its learned weights, which descriptor streams actually carry signal.